

WebRTC, LiveKit and Janus: 제품별 통제 범위에 맞춘 선택

LiveKit와 Janus는 우열의 문제가 아니라, 제품별로 직접 소유해야 하는 통제 범위가 다른 문제였다.

요약

원격제어·원격기술지원 제품에는 LiveKit를 사용했다. 원격 화상회의 제품에는 Janus를 선택했다. 두 선택은 서로 충돌하지 않는다. 원격제어에서는 room, participant, track, SDK, token 기반의 생산성과 명확한 추상화가 중요했다. 화상회의에서는 media gateway 내부 흐름, session lifecycle, plugin 경계, RTP/RTCP 관측, 장애 분석 가능성이 더 중요했다.

핵심 판단은 특정 솔루션의 우열이 아니다. 제품마다 직접 소유해야 할 제어면(control plane)이 달랐고, 그 차이에 맞춰 도구를 분리했다.

선택을 하나로 묶지 않은 이유

WebRTC 제품을 하나의 기준으로 평가하면 판단이 흐려진다. 원격제어와 원격 화상회의는 모두 실시간 미디어를 사용하지만 운영 책임이 다르다.

구분	원격제어·원격기술지원	원격 화상회의
적용 도구	LiveKit	Janus
핵심 목표	상담원과 고객을 빠르게 연결하고 화면 공유·제어 승인 흐름을 안정화한다.	다자간 media session과 gateway 내부 흐름을 직접 설명한다.
중요한 추상화	room, participant, track, client SDK, token, session API	session, handle, plugin, RTP/RTCP, transport, media forwarding
장애 관점	고객 화면 공유 끊김, 제어 요청 실패, 상담 흐름 중단	ICE/DTLS/SRTP, RTP 흐름, RTCP feedback, publisher/subscriber 상태
운영 책임	상담 이력, 권한 승인, 연결 상태, 고객 응대 흐름	media gateway 제어면, plugin 책임 경계, media path 관측

이 차이를 명시하지 않으면 “어떤 WebRTC 서버가 더 나은가”라는 단순 비교로 흐른다. 실제 판단 기준은 제품이 요구하는 통제 범위다.

LiveKit를 원격제어에 사용한 이유

LiveKit는 원격제어·원격기술지원 제품에 적합했다. 상담원과 고객이 세션에 입장하고, 화면 공유가 시작되며, 필요한 경우 제어 요청과 승인 흐름이 이어진다. 이 영역에서는 낮은 수준의 media server 내부 구현보다 제품 기능을 빠르게 안정화할 수 있는 추상화가 중요했다.

LiveKit의 room, participant, track, token, server SDK 모델은 이 흐름과 잘 맞았다. application layer는 상담 권한, 고객사 정책, 이력, 제어 승인, audit log에 집중하고, media session의 기본 구성은 플랫폼 추상화에 맡길 수 있었다.

따라서 LiveKit를 선택한 이유는 단순한 편의가 아니다. 원격제어 제품에서 직접 구현해야 할 영역과 플랫폼에 위임해도 되는 영역을 분리했을 때, LiveKit의 추상화가 제품 요구와 맞았다.

Janus를 원격 화상회의에 사용한 이유

Janus는 원격 화상회의 제품에 사용했다. 화상회의에서는 room abstraction만으로 운영 요구를 설명하기 어려웠다. publisher와 subscriber가 늘어나고, media forwarding, RTP/RTCP feedback, plugin event, session lifecycle, 장애 추적 지점이 운영 품질에 직접 영향을 준다.

Janus를 선택한 기준은 내부 책임 경계를 검증할 수 있는가였다. Janus는 plugin 구조가 명확하고, session, handle, transport, plugin, RTP/RTCP 처리 흐름을 코드 기준으로 추적할 수 있다. 문서와 샘플 설정만으로 판단하지 않고 실행 흐름을 확인할 수 있다는 점이 중요했다.

초기 검증 대상은 설정 파일이 아니라 runtime 흐름이었다.

```
transport request
-> session create
-> handle attach
-> plugin request
-> SDP offer / answer
-> ICE / DTLS / SRTP
-> RTP / RTCP media path
-> application audit / meeting policy
```

이 흐름을 확인하면서 WebRTC Gateway와 application server의 책임을 분리했다. Gateway는 media session과 plugin lifecycle을 책임지고, application layer는 회의 권한, 조직 정책, 감사 로그, 회의 이력을 책

임진다.

Wowza 같은 상용 솔루션을 바라본 기준

Wowza 같은 상용 솔루션은 안정성, 기술지원, 운영 문서, 상용 지원 체계에서 장점이 있다. 조직이 media server 내부 구조보다 서비스 운영과 고객 지원에 집중해야 한다면 합리적인 선택이 될 수 있다.

다만 원격 화상회의 영역에서는 제품 내부의 제어면을 직접 소유하는 것이 더 중요했다. 세션 생성 조건, 회의 참여 권한, signaling 흐름, media event, 장애 trace, plugin 확장 지점을 제품 정책과 함께 설계해야 했다. 이 범위를 상용 제품의 추상화에 맞추기보다, 내부 구조를 읽고 필요한 지점만 제한적으로 확장하는 쪽이 장기 운영 기준과 맞았다.

Janus는 쉬운 선택이 아니라 더 많은 책임을 감수하는 선택이었다. 대신 화상회의 제품의 핵심 제어면을 외부 솔루션에 위임하지 않고, 권한·세션·감사·장애 대응 구조와 함께 설계했다.

RTP/RTCP를 기준으로 본 화상회의

WebRTC를 API 중심으로 접근하면 `getUserMedia`, `RTCPeerConnection`, `createOffer`, `setRemoteDescription` 호출 순서가 먼저 드러난다. 그러나 운영에서 더 중요한 것은 연결 이후다. media가 어떻게 흐르고, 품질이 나빠졌을 때 어떤 feedback이 오며, 그 feedback을 서비스 정책과 어떻게 연결할 것인가가 핵심이다.

RTP는 media data를 운반하고, RTCP는 품질과 상태에 대한 feedback을 제공한다. 이 둘을 이해하지 못하면 영상 끊김이 network 문제인지, encoder 설정 문제인지, subscriber 증가에 따른 forwarding 문제인지, plugin 처리 지연인지 판단하기 어렵다.

화상회의 장애 분석은 session 상태, ICE 상태, RTP 흐름, RTCP feedback, plugin event, application audit 이 연결되어 있어야 한다. 이 기준이 Janus 검증의 핵심이었다.

Plugin을 직접 만든 이유

Janus plugin을 직접 구현한 목적은 기능 확장이 아니라 확장 지점과 실패 영향 범위를 확인하는 것이었다. Gateway core와 plugin의 책임은 다르다. core는 session, transport, media path, plugin lifecycle을 안정적으로 유지해야 한다. plugin은 media 흐름과 직접 관련된 제어만 제한적으로 담당해야 한다.

반면 조직 정책, 회의 업무 상태, 권한 승인, 이력 저장, 알림, 외부 시스템 연동은 application layer가 담당하는 편이 안전하다. plugin에 업무 정책을 과하게 넣으면 Gateway가 application server가 되고, 장애 영향 범위가 커진다.

LiveKit, Janus, mediasoup, Wowza를 바라보는 기준

도구의 이름보다 제품이 요구하는 통제 범위가 중요하다.

- 원격제어·원격기술지원처럼 빠른 연결, SDK 생산성, room 기반 session 흐름이 중요하다면 LiveKit가 적합하다.
- 운영 지원과 상용 기술지원이 핵심이면 Wowza 계열 상용 솔루션이 적합할 수 있다.
- JavaScript/TypeScript 중심으로 media server를 조합해야 한다면 mediasoup이 자연스럽다.
- C 기반 core와 plugin 구조를 읽고 media session 경계를 직접 검증해야 한다면 Janus가 적합하다.

이 프로젝트의 결론은 Janus의 우월성을 주장하는 것이 아니다. 원격제어에서는 LiveKit를 사용했고, 원격 화상회의에서는 Janus를 선택했다. 제품별로 직접 소유해야 하는 통제 범위가 달랐고, 그 차이에 맞춰 도구를 분리했다.

시스템 설계자의 그림

WebRTC 선택은 media server 비교가 아니라 제품별 제어면 분리 문제였다. 원격제어에서는 상담 권한, 동의, 세션 이력 같은 application 통제가 중요했고, 화상회의에서는 gateway 내부 session, plugin, RTP/RTCP 관측 지점이 운영 품질을 좌우했다.

버린 선택과 이유

원격제어와 화상회의를 하나의 WebRTC 플랫폼으로 통합하는 방식은 단순해 보이지만, 제품별 책임 경계를 흐릴 수 있었다. 원격제어는 상담 흐름, 승인, 화면 공유 안정성이 핵심이고, 화상회의는 다자간 media path와 gateway 제어면이 핵심이다. 같은 기술 범주라도 운영 기준은 달랐다.

상용 솔루션에 모든 media 책임을 위임하는 방식도 검토 대상이었지만, 화상회의 영역에서는 내부 session lifecycle과 plugin event를 제품 정책과 연결해야 했다. 장애 시 ICE, DTLS, SRTP, RTP, RTCP, publisher/subscriber 상태를 분리해서 볼 수 있어야 했기 때문이다.

반대로 모든 WebRTC 계층을 직접 구현하는 방식도 선택하지 않았다. 원격제어 제품에서는 LiveKit의 추상화가 충분히 제품 책임과 맞았고, 직접 구현은 운영 리스크와 개발 비용만 키울 수 있었다.

운영에서 확인해야 하는 media 흔적

흔적	목적
session/handle lifecycle	회의 참여와 media handle 상태를 추적한다.

흔적	목적
ICE/DTLS/SRTP 상태	연결 실패와 암호화/media path 문제를 분리한다.
RTP packet 흐름	media forwarding 지연과 손실을 판단한다.
RTCP feedback	품질 저하 원인을 network, encoder, subscriber 상태와 연결한다.
application audit	회의 권한, 조직 정책, 참여 이력을 media event와 연결한다.

설계 결론

WebRTC 제품에서 media server는 단순 인프라가 아니다. session, 권한, signaling, media path, audit, 장애 대응이 만나는 제어면이다. 더 빠르게 만들 수 있는 선택과 더 깊게 통제해야 하는 선택을 구분해야 한다. LiveKit와 Janus의 선택은 그 구분을 제품별로 적용한 결과다.